

What it does `renderCrossFileCallEdges` takes an `outline` object and produces an HTML fragment that lists cross-file call edges. It extracts `outline.cross_file_call_edges` (or defaults to an empty array), then iterates over each edge, escaping the `from_file`, `to_file`, and `via` properties with `escapeHtml`. Up to 90 rows are rendered as `<tr>` elements inside a table. If no rows are produced, it returns a paragraph indicating that no unambiguous edges were found.

How it works 1. Edge extraction –

`Array.isArray(outline?.cross_file_call_edges)` guards against missing or non-array values. 2. **Loop with cap*

- `for (const e of edges)` iterates, breaking after 90 iterations (`if (n >= 90) break`). 3.
**Escaping & row building*
- Each field is escaped (`escapeHtml(e?.from_file ?? "")`) and concatenated into `rows`. 4.
**Conditional output*
- If `rows` is empty, a fallback `<p>` is returned; otherwise a `<table>` with headers and the accumulated rows is returned.

Notes for reviewers– `escapeHtml` is referenced but not defined in this snippet; ensure it's imported or available in scope. – The 90-row limit is hard-coded; consider making it configurable if the dataset can grow. – The function assumes `outline` may be `null` or `undefined`; the optional chaining (`outline?.`) handles this. – The fallback message uses a heuristic label; verify that this aligns with the intended UX. – No error handling for malformed edge objects beyond defaulting to empty strings; additional validation could improve robustness.